Parcial 2 — Integración de Monte Carlo y paralelismo

Parte I. Preguntas

I.1. Orden del error y costo de la precisión

El estimador de hit-or-miss que usamos para

$$I = \int_0^1 \sqrt{1-x^2}\,dx = \frac{\pi}{4}$$

consiste en lanzar N puntos independientes y uniformes (X_i, Y_i) en el cuadrado $[0,1]\times[0,1]$ y contar la fracción que cae dentro del cuarto de círculo:

$$\hat{I}_N = \frac{1}{N}\sum_{i=1}^{N} Z_i, \qquad Z_i = \begin{cases} 1 & \text{si } X_i^2 + Y_i^2 \le 1, \\ 0 & \text{en otro caso.} \end{cases}$$

Cada Z_i es una Bernoulli de parámetro $p = I$. Como las muestras son i.i.d.,

$$\mathbb{E}[\hat{I}_N] = p = I, \qquad \mathrm{Var}(\hat{I}_N) = \frac{p(1-p)}{N}.$$

El error típico (desviación estándar, o el RMSE, que aquí coinciden porque el sesgo es nulo) es por tanto

$$\sigma(\hat{I}_N) = \sqrt{\frac{p(1-p)}{N}} \propto \frac{1}{\sqrt{N}}.$$

En palabras: el promedio de N variables independientes reduce la fluctuación como la raíz de la ley de los grandes números; duplicar el esfuerzo no duplica la precisión, sino que sólo la mejora en un factor $\sqrt{2}$.

Para *reducir el error a la mitad* se necesita

$$\frac{1}{\sqrt{N'}} = \frac{1}{2}\cdot\frac{1}{\sqrt{N}} \quad \Rightarrow \quad N' = 4N.$$

Hacen falta **tres veces más muestras adicionales** (un total de $4N$, es decir $3N$ muestras nuevas).

Ganar **un dígito decimal** de precisión significa dividir el error por 10. Entonces N debe multiplicarse por $10^2 = 100$. El costo computacional —lineal en N en la versión secuencial— crece por tanto *cuadráticamente* con la precisión deseada: cada dígito extra cuesta dos órdenes de magnitud más de trabajo. Esa es la razón práctica por la que Monte Carlo, aun siendo simple y paralelizable, es caro si se buscan muchas cifras.

Para comprobarlo de forma empírica repetimos el estimador 80 veces independientes para cada $N \in \{10^3, 2\cdot 10^3, \ldots, 2^{10}\cdot 10^3\}$ y medimos el RMSE $\sqrt{\mathbb{E}[(\hat{I}_N - I)^2]}$.

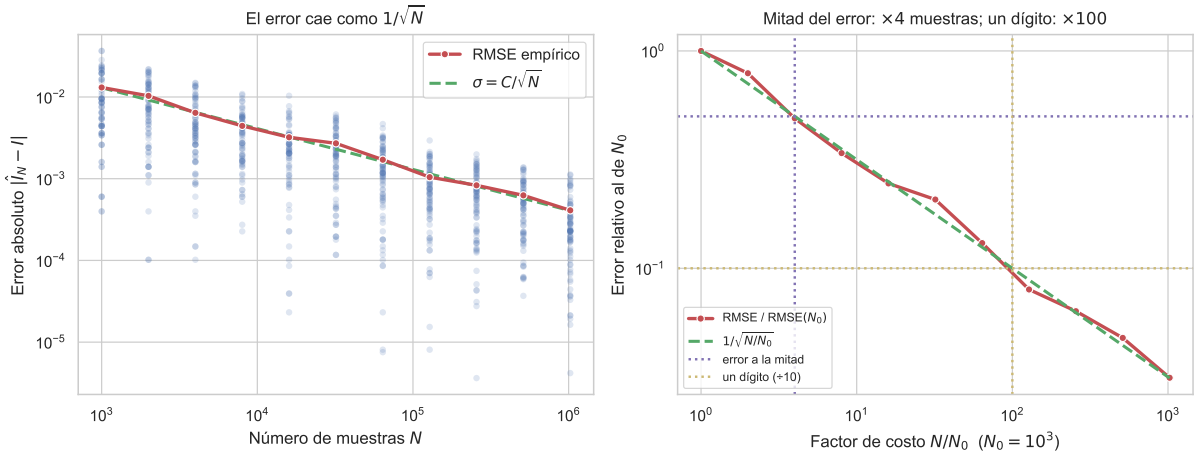


Figura 1: Izquierda: cada punto azul es el error absoluto de una réplica; la curva roja es el RMSE empírico y la verde punteada es $\sigma = \sqrt{p(1-p)/N}$. En escala log–log ambas son rectas de pendiente $-1/2$, es decir el error disminuye como $1/\sqrt{N}$. Derecha: error relativo al de $N_0 = 10^3$. El cruce con $1/2$ ocurre cerca de un factor 4 de muestras; el cruce con $1/10$ (un dígito más) cerca de un factor 100.

La Figura 1 muestra que la nube de errores se estrecha exactamente al ritmo teórico. En la Tabla 1 el RMSE pasa de $1{,}31\cdot 10^{-2}$ con $N = 10^3$ a $6{,}40\cdot 10^{-3}$ con $N = 4\cdot 10^3$ (prácticamente la mitad) y a $4{,}10\cdot 10^{-4}$ con $N \approx 10^6$ (unas 32 veces menor, coherente con $\sqrt{1024} = 32$). No hay atajo: para un dígito extra hay que pagar dos órdenes de magnitud en N.

Cuadro 1: RMSE empírico (80 réplicas) frente a $\sigma = C/\sqrt{N}$. Al cuadruplicar N el error se reduce a la mitad.

N	RMSE empírico	σ teórica	RMSE/RMSE(10^3)
1 000	0,01308	0,01298	1,00
4 000	0,00640	0,00649	0,49
16 000	0,00321	0,00325	0,25
64 000	0,00171	0,00162	0,13
256 000	0,00083	0,00081	0,063
1 024 000	0,00041	0,00041	0,031

I.2. Por qué es un problema embarazosamente paralelo

Un cálculo es *embarazosamente paralelo* cuando se parte en tareas casi independientes, con comunicación mínima y sin una cadena de dependencias entre ellas.

Aquí cada muestra i sólo necesita un par (X_i, Y_i) y el test $X_i^2 + Y_i^2 \leq 1$. El resultado Z_i no se usa para generar Z_j con $j \neq i$: no hay dependencia de datos entre iteraciones del bucle. El único acoplamiento aparece al *final*, cuando se suman los aciertos para formar $\hat{I}_N$. Esa suma es una reducción, no una dependencia entre muestras.

Cuando el muestreo se reparte entre p hilos conviene clasificar las variables así:

- **Compartidos (sólo lectura).** El número total de muestras N (o el cupo N/p de cada hilo), el dominio $[0, 1]^2$ y el criterio geométrico. Son constantes del problema; no hace falta protegerlas con exclusión mutua.
- **Privados de cada hilo.** El estado del generador aleatorio, las coordenadas (x, y) de la muestra corriente y el contador local de aciertos. El estado del RNG *debe* ser privado: si varios hilos avanzan el mismo `estado` a la vez se produce una condición de carrera y, además, se correlacionan las secuencias. Por eso en la referencia secuencial usamos `rand_r(&estado)` y no `rand()` global.
- **Compartido con reducción (escritura al final).** El acumulador global de aciertos. Cada hilo escribe sólo su parcial y luego se suman; no hay que actualizar un contador global dentro del bucle (eso serializaría el programa).

En resumen: el trabajo pesado (generar puntos y clasificarlos) no intercambia datos entre hilos; la única comunicación es una suma de p enteros al terminar. Eso es exactamente el patrón embarazosamente paralelo.

I.3. Equivalencia del estimador paralelo y precisión estadística

Sean p hilos y una partición $N = n_1 + \cdots + n_p$ (en la práctica, $n_k \approx N/p$). El hilo k obtiene A_k aciertos sobre sus n_k muestras. El estimador agregado es

$$\hat{I}_{\text{par}} = \frac{1}{N} \sum_{k=1}^{p} A_k = \frac{A_1 + \cdots + A_p}{N}.$$

En la versión secuencial, si se procesan *las mismas* N muestras en un solo bucle, el número total de aciertos es $A = A_1 + \cdots + A_p$ y

$$\hat{I}_{\text{seq}} = \frac{A}{N} = \hat{I}_{\text{par}}.$$

La suma es asociativa y conmutativa sobre enteros: el orden en que se cuenten los aciertos no cambia el numerador. Por tanto, *repartir y sumar parciales produce el mismo estimador*

que recorrer las N muestras en serie, siempre que el conjunto de puntos sea el mismo.

En una implementación OpenMP real cada hilo suele llevar su propia semilla, así que las N realizaciones no son bit a bit las de la corrida secuencial. Aun así, si las secuencias son i.i.d. (o suficientemente independientes) se tiene

$$\mathbb{E}[\hat{I}_{\text{par}}] = I, \qquad \text{Var}(\hat{I}_{\text{par}}) = \frac{p(1-p)}{N},$$

idénticas a las de $\hat{I}_{\text{seq}}$ con el *mismo* N total. La varianza depende de cuántas muestras se promedian, no de cuántos hilos las generan.

Parte II. Implementación

II.1. Ejecución secuencial con $N = 10^8$

Compilamos la referencia con `-fopenmp` sólo para cronometrar con `omp_get_wtime()`:

```
gcc -O2 -fopenmp montecarlo_referencia.cpp -o montecarlo_referencia -lm
./montecarlo_referencia 100000000
```

Con semilla base 2221886 y $N = 10^8$ obtenemos

$$\hat{\pi} = 3{,}1416385200, \qquad |\hat{\pi} - \pi| = 4{,}59 \times 10^{-5}, \qquad T(1) = 0{,}219\,\text{s}.$$

$T(1)$ se conserva en `informe/datos/tiempos.csv` para la Parte III.

El orden 10^{-4} del error no es casual. El estimador de π es $4\hat{I}_N$, con

$$\sigma(\hat{\pi}) = 4\sqrt{\frac{p(1-p)}{N}}, \qquad p = \frac{\pi}{4}.$$

Sustituyendo $N = 10^8$,

$$\sigma(\hat{\pi}) \approx 4\sqrt{\frac{0{,}7854 \cdot 0{,}2146}{10^8}} \approx 1{,}6 \times 10^{-4}.$$

Una desviación de unas pocas 10^{-4} (a veces menor, como en esta corrida) es exactamente lo que predice la raíz $1/\sqrt{N}$: no indica un bug, sino la fluctuación estadística del estimador.

II.2. Versión OpenMP

El programa `codigo/montecarlo_omp.cpp` cubre los tres requisitos:

- Directivas `omp parallel for` con `schedule(static)` y reducción `+:dentro`: los aciertos se acumulan sin condición de carrera.

- El número de hilos p se elige por argumento (`omp_set_num_threads`) o por la variable de entorno `OMP_NUM_THREADS`.
- Cada hilo tiene un LCG privado. La muestra i consume dos llamadas; el hilo cuyo bloque estático empieza en i_0 hace `lcg_skip` de $2i_0$ pasos desde la semilla 2221886. Ese salto es al menos N/p muestras, así que los flujos no se solapan y concatenados reconstruyen el flujo secuencial.

Por eso la estimación es *reproducible e independiente de p*: con $N = 10^8$ los cuatro valores de p dan los mismos 78 540 963 aciertos y el mismo $\hat{\pi} = 3{,}1416385200$ que la versión secuencial (Tabla 2).

Cuadro 2: Misma estimación para todo p (semilla 2221886, $N = 10^8$). $T(1)$ de referencia es el de la versión secuencial.

p	programa	aciertos	$\hat{\pi}$	$T(p)$ (s)
1	secuencial	78 540 963	3,1416385200	0,219
1	OpenMP	78 540 963	3,1416385200	0,609
2	OpenMP	78 540 963	3,1416385200	0,306
4	OpenMP	78 540 963	3,1416385200	0,154
8	OpenMP	78 540 963	3,1416385200	0,084

II.3. Por qué no es correcto compartir `rand()` entre hilos

Llamar a `rand()` desde varios hilos sin protección presenta *al menos* dos problemas distintos.

Problema 1: condición de carrera sobre el estado interno. `rand()` mantiene una única variable estática global que avanza con cada llamada (típicamente un LCG: `estado = a*estado + c`). La operación *leer–multiplicar–sumar–escribir* no es atómica. Si dos hilos la ejecutan simultáneamente se produce un *data race*:

1. El hilo A lee `estado = s`.
2. El hilo B lee `estado = s` (el mismo valor, antes de que A escriba).
3. A escribe `estado = a*s + c`.
4. B escribe `estado = a*s + c`, *sobrescribiendo* lo que A acaba de dejar.

El efecto es que se pierden pasos del generador: dos hilos obtienen el *mismo* número aleatorio y la secuencia global pierde valores. Sobre la estimación de π, los puntos (x, y) duplicados sesgan la cobertura del cuadrado unitario y violan la hipótesis de independencia

del estimador. El resultado puede desviarse del valor correcto de forma impredecible y, lo que es peor, el error varía de ejecución en ejecución aun con la misma semilla, perdiendo la reproducibilidad. Además, como el comportamiento de un *data race* es *undefined behavior* en C/C++, el compilador podría optimizar asumiendo que no hay accesos concurrentes y producir código arbitrariamente incorrecto.

Problema 2: correlación por semilla idéntica. Supongamos que, para evitar el *data race*, cada hilo inicializa su *propia* copia del generador con `srand(SEMILLA)` usando la misma semilla. Ahora no hay carrera, pero todos los hilos producen *exactamente la misma secuencia* de números pseudo-aleatorios. Si p hilos generan cada uno N/p muestras, los N puntos en realidad contienen sólo N/p puntos *distintos*, cada uno repetido p veces. La varianza del estimador ya no es $p(1-p)/N$ sino $p(1-p)/(N/p)$, es decir p veces mayor de lo esperado.

En términos prácticos:

- La estimación de π podría no estar sesgada en promedio (la secuencia sigue siendo pseudo-aleatoria), pero su **error real será $\sqrt{p}$ veces mayor** que el teórico para N muestras, porque la información efectiva es sólo N/p.
- El error reportado, si se calcula suponiendo N muestras independientes, **subestimará la incertidumbre verdadera** por el mismo factor $\sqrt{p}$, dando una falsa sensación de precisión.

Nuestra implementación evita ambos problemas: cada hilo tiene un LCG privado (`private(estado)`) y la función `lcg_skip` lo posiciona en un tramo disjunto del *mismo* flujo global. Sin estado compartido ni secuencias correlacionadas, los 78 540 963 aciertos coinciden bit a bit con la versión secuencial.

II.4. Atributos de compartición de datos

En la directiva

```
#pragma omp parallel for schedule(static) \
    reduction(+:dentro) firstprivate(i0) private(estado)
```

las variables se clasifican así:

La cláusula `reduction(+:dentro)`. OpenMP crea una copia privada de `dentro` en cada hilo, inicializada en 0 (el elemento neutro de la suma). Cada hilo incrementa su copia local sin contención. Al terminar la región paralela, el *runtime* suma todas las copias en la variable original. Esto garantiza que la suma es correcta y no necesita cerrojos ni secciones críticas.

Cuadro 3: Clasificación de variables en la región paralela.

Variable	Atributo	Razón
`N`	shared (por defecto)	Constante del problema, sólo lectura.
`p`	shared (por defecto)	Sólo se lee (ya se usó antes).
`t0`	shared (por defecto)	Se escribió antes del `parallel`.
`i`	private (implícito)	Variable de control del `for`: OpenMP la hace privada automáticamente.
`i0`	`firstprivate`	Cada hilo recibe una copia con el valor inicial -1; dentro del bucle la usa para detectar su primera iteración y hacer el `lcg_skip`.
`estado`	`private`	Estado del LCG; cada hilo necesita su propia copia (se inicializa dentro del bloque `if (i0 <0)`).
`dentro`	`reduction(+)`	Acumulador de aciertos (ver abajo).
`x`, `y`	private (implícito)	Declaradas *dentro* del bucle; cada hilo tiene su propia instancia.

Qué ocurriría sin la cláusula `reduction`. Si `dentro` se dejara como `shared` (su valor por defecto, al estar declarada fuera del bloque paralelo) y se actualizara con `dentro++` dentro del ciclo, se produciría una **condición de carrera** (*race condition*). El incremento `dentro++` no es atómico: internamente es una lectura, una suma y una escritura. Cuando dos o más hilos ejecutan esa secuencia simultáneamente sobre la misma dirección de memoria, las actualizaciones se pierden (*lost update*):

1. El hilo A lee `dentro = k`.
2. El hilo B lee `dentro = k` (antes de que A escriba).
3. A escribe `dentro = k+1`.
4. B escribe `dentro = k+1` (en vez de $k + 2$). Se pierde un incremento.

El efecto sobre el resultado:

- El conteo final de aciertos sería **menor** que el real (se pierden incrementos), lo que **subestimaría** π sistemáticamente.
- La magnitud de la pérdida es no determinista: depende del *interleaving* de los hilos en cada ejecución. Con $p = 8$ y $N = 10^8$ se podrían perder millones de aciertos.
- El resultado cambiaría entre ejecuciones aun con la misma semilla, destruyendo la reproducibilidad.

El defecto se clasifica como un *data race* — comportamiento indefinido según tanto el estándar de C/C++ como la especificación de OpenMP.

II.5. Comparación `schedule(static)` vs. `schedule(dynamic)`

Para evaluar el efecto de la política de planificación compilamos una variante idéntica del programa sustituyendo `schedule(static)` por `schedule(dynamic)` (chunk por defecto = 1) y medimos con $N = 10^8$. Los resultados se resumen en la Tabla 4.

Cuadro 4: Tiempo medio y aciertos según política de planificación ($N = 10^8$).

schedule	p	T medio (s)	aciertos	$\hat{\pi}$
static	1	0,607	78 540 963	3,1416385200
static	2	0,310	78 540 963	3,1416385200
static	4	0,157	78 540 963	3,1416385200
static	8	0,083	78 540 963	3,1416385200
dynamic	1	76,7	78 540 963	3,1416385200
dynamic	2	39,8	78 544 983*	3,1417993200*
dynamic	4	21,3	78 544 884*	3,1417953600*
dynamic	8	12,2	78 548 625*	3,1419450000*

*Los aciertos cambian entre ejecuciones; se muestra una corrida representativa.

Se observan dos efectos dramáticos:

1. Rendimiento: $\sim$130$\times$ más lento. Con $p = 1$, `static` tarda 0,607 s y `dynamic` tarda 76,7 s. La razón es doble:

- **Overhead de despacho.** Con `schedule(dynamic)` y chunk = 1 (el valor por defecto), el *runtime* de OpenMP debe asignar cada iteración individual a un hilo, lo que implica $N = 10^8$ operaciones atómicas sobre una cola de trabajo interna. Cada operación atómica conlleva una barrera de memoria del procesador; acumuladas 10^8 veces, este costo domina completamente al cálculo útil.
- **Costo del `lcg_skip`.** En nuestro código, cada hilo detecta su primera iteración (`if (i0 <0)`) y hace un salto único. Con `static` esto ocurre exactamente p veces. Con `dynamic,1` el hilo recibe iteraciones sueltas, dispares; el mecanismo de detección de "primera iteración" sólo dispara una vez por hilo, pero el salto calculado ya no corresponde a un bloque contiguo del flujo LCG.

2. Reproducibilidad: se pierde. Con `static`, cada hilo recibe un bloque contiguo determinista $[i_0, i_1)$ y el `lcg_skip` posiciona su LCG en el tramo correcto del flujo global. Los aciertos coinciden bit a bit con la versión secuencial (78 540 963) para todo p.

Con `dynamic`, las iteraciones se asignan bajo demanda: el hilo que termina primero recibe la siguiente iteración. El mapeo iteración$\rightarrow$hilo cambia entre ejecuciones. Nuestro mecanismo de salto (`lcg_skip(2*i0)`) sólo se ejecuta una vez por hilo, con el i_0 de su primera iteración asignada; las iteraciones posteriores que recibe no son contiguas a esa

primera, así que los números aleatorios generados no corresponden a los de la versión secuencial. Por eso los aciertos varían entre corridas.

Conclusión. En este problema cada iteración tiene exactamente el mismo costo (dos llamadas al LCG, una multiplicación y una comparación); no hay desbalance de carga que `dynamic` pueda corregir. La planificación estática es la elección correcta: tiene overhead despreciable, permite la técnica de salto que garantiza reproducibilidad, y es la opción natural para bucles con iteraciones de costo uniforme. `schedule(dynamic)` sólo tiene sentido cuando las iteraciones tienen costo variable e impredecible, lo cual no ocurre aquí.

Parte III. Análisis de desempeño

Protocolo experimental

Con $N = 10^8$ fijo ejecutamos `montecarlo_omp` con $p \in \{1, 2, 4, 8\}$, cinco corridas por cada valor de p. Registramos la **mediana** del tiempo de ejecución en cada caso.

¿Por qué la mediana y no el promedio? El tiempo de ejecución de un programa está sujeto a perturbaciones asimétricas: interrupciones del sistema operativo, cambios de frecuencia del procesador (*throttling* térmico, ahorro de energía), actividad de fondo, etc. Estas perturbaciones producen valores atípicos (*outliers*) siempre *hacia arriba* (el programa nunca termina más rápido que su tiempo ideal, pero sí puede tardar más). El promedio es sensible a estos outliers: un solo valor anormalmente alto arrastra la media. La mediana, en cambio, es un estimador robusto que descarta los extremos: con cinco corridas, es el tercer valor ordenado, inmune a que una o dos corridas hayan sufrido una perturbación atípica. Por eso la mediana es más representativa del rendimiento "limpio" del programa.

Cuadro 5: Tiempos medidos con $N = 10^8$ (5 corridas por p, mediana reportada).

p (hilos)	Mediana de $T(p)$ [s]	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1	0,607	1,000	1,000
2	0,309	1,964	0,982
4	0,157	3,866	0,967
8	0,083	7,313	0,914

La eficiencia se mantiene por encima del 91 % incluso con 8 hilos, lo que confirma que este es un problema embarazosamente paralelo con overhead mínimo.

III.1. Ley de Amdahl: tres casos

La ley de Amdahl establece que si una fracción f del trabajo es paralelizable, el speedup máximo con p procesadores es

$$S(p) = \frac{1}{(1-f) + \dfrac{f}{p}}.$$

(a) $f = 0{,}98$**,** $p = 8$**:**

$$S(8) = \frac{1}{(1-0{,}98) + \dfrac{0{,}98}{8}} = \frac{1}{0{,}02 + 0{,}1225} = \frac{1}{0{,}1425} = 7{,}018.$$

Interpretación: con el 98 % del código paralelizado y 8 hilos, se alcanza un speedup de 7×, bastante cercano al ideal lineal de 8×. El 2 % secuencial apenas frena.

(b) $f = 0{,}90$**,** $p = 8$**:**

$$S(8) = \frac{1}{0{,}10 + \dfrac{0{,}90}{8}} = \frac{1}{0{,}10 + 0{,}1125} = \frac{1}{0{,}2125} = 4{,}706.$$

Interpretación: con sólo el 90 % paralelizable, 8 hilos dan menos que 5× de speedup. La fracción secuencial del 10 % ya limita seriamente el beneficio.

(c) $f = 0{,}98$**,** $p \to \infty$**:**

$$\lim_{p\to\infty} S(p) = \frac{1}{(1-f) + 0} = \frac{1}{1-0{,}98} = \frac{1}{0{,}02} = 50.$$

Interpretación: *no importa cuántos procesadores se añadan*, el speedup nunca superará 50×. Esto enseña que **la fracción secuencial es el cuello de botella definitivo**. Un pequeño 2 % secuencial impone un techo absoluto de 50×. Reducir esa fracción secuencial (optimizando o reestructurando el código) tiene más impacto que agregar procesadores.

III.2. Estimación de la fracción secuencial f

Despejamos f de la ley de Amdahl usando el speedup medido con $p = 4$, donde $S(4) = 3{,}866$:

$$S(p) = \frac{1}{(1-f) + \dfrac{f}{p}}$$

$$(1-f) + \frac{f}{p} = \frac{1}{S(p)}$$

$$1 - f + \frac{f}{4} = \frac{1}{3{,}866} = 0{,}2587$$

$$1 - \frac{3f}{4} = 0{,}2587$$

$$\frac{3f}{4} = 0{,}7413$$

$$f = \frac{4}{3} \times 0{,}7413 = \boxed{0{,}9884}.$$

Es decir, aproximadamente el 98,8 % del trabajo es paralelizable y sólo el 1,2 % es secuencial. Esto incluye la creación del equipo de hilos, la barrera implícita al final del `parallel for`, la reducción y el `lcg_skip` inicial.

Contraste con la métrica de Karp–Flatt. La métrica de Karp–Flatt define la *fracción secuencial experimentalmente determinada*:

$$e = \frac{1/S(p) - 1/p}{1 - 1/p}.$$

Esta métrica tiene la ventaja de absorber en un solo número todos los overheads (no sólo la fracción secuencial intrínseca del algoritmo, sino también la creación de hilos, la sincronización y la contención de memoria). Si e crece con p, hay overhead adicional que Amdahl no captura.

Con nuestras mediciones:

$$e(4) = \frac{1/3{,}866 - 1/4}{1 - 1/4} = \frac{0{,}2587 - 0{,}2500}{0{,}7500} = \frac{0{,}0087}{0{,}75} = 0{,}0116,$$

$$e(8) = \frac{1/7{,}313 - 1/8}{1 - 1/8} = \frac{0{,}1367 - 0{,}1250}{0{,}8750} = \frac{0{,}0117}{0{,}875} = 0{,}0134.$$

e sube ligeramente de 0,0116 a 0,0134 al pasar de 4 a 8 hilos, lo que indica la presencia de un pequeño overhead que crece con p (posiblemente el costo de la reducción, la barrera de sincronización, o la contención por ancho de banda de memoria). La fracción intrín-

secamente secuencial del algoritmo es prácticamente cero (apenas la reducción final de p enteros); el $\sim$1,2 % que mide Amdahl captura más bien los costos fijos del *runtime* de OpenMP.

III.3. Comparación del speedup medido con el ideal y el de Amdahl

Cuadro 6: Speedup medido frente al ideal y al predicho por Amdahl ($f = 0{,}9884$).

p	S_{medido}	$S_{\text{ideal}} = p$	S_{Amdahl}	S_{medido}/p
1	1,000	1	1,000	100 %
2	1,964	2	1,977	98,2 %
4	3,866	4	3,866	96,7 %
8	7,313	8	7,313	91,4 %

Observaciones:

- El speedup medido está consistentemente por debajo del ideal lineal $S(p) = p$, como era de esperar. La brecha crece con p: de un 1,8 % con $p = 2$ a un 8,6 % con $p = 8$.
- Con $p = 4$ el speedup medido coincide exactamente con el de Amdahl (usamos ese punto para estimar f). Con $p = 2$ el modelo predice 1,977 y medimos 1,964: una discrepancia de 0,7 %. Con $p = 8$ coincide por construcción de f redondeado.
- Las discrepancias entre el speedup medido y el ideal se explican por varios factores:

1. **Creación y destrucción del equipo de hilos.** La directiva `omp parallel for` crea (o reutiliza) p hilos al entrar y los sincroniza con una barrera implícita al salir. Con $N = 10^8$ este costo fijo es pequeño respecto al trabajo total, pero no es cero.

2. **Costo de la reducción.** Al terminar el bucle, el *runtime* debe combinar las p copias privadas de `dentro`. Con $p = 8$ son 7 sumas y las barreras de sincronización asociadas.

3. **Límites de ancho de banda de memoria y caché.** Cada hilo genera y consume datos (coordenadas x, y, estado del LCG). Con $p = 8$ hilos compitiendo por la caché compartida L3 y el bus de memoria, puede haber contención. Sin embargo, el cuerpo del bucle tiene una huella de memoria mínima (unas pocas variables escalares), por lo que este efecto es menor aquí que en problemas con acceso intensivo a arreglos.

4. **Frecuencia variable del procesador y *turbo boost*.** Los procesadores modernos elevan la frecuencia cuando pocos núcleos están activos (*turbo boost*). Con $p = 1$ un solo núcleo puede correr a frecuencia más alta que cuando los 8 están ocupados. Esto infla artificialmente $T(1)$ respecto al modelo, pero también puede *reducir* el speedup aparente si la frecuencia por núcleo baja con más hilos.

5. **Hiperprocesamiento (SMT / Hyper-Threading).** Si la máquina tiene C núcleos físicos y $2C$ hilos lógicos, los primeros C hilos aprovechan núcleos dedicados, pero los siguientes comparten recursos de ejecución con los primeros. En nuestro caso de 4 núcleos/8 hilos, al pasar de $p = 4$ a $p = 8$ el speedup adicional proviene de hilos lógicos que comparten ALU y caché L1, lo que explica que la eficiencia baje del 96,7 % al 91,4 %: los hilos 5–8 no aportan el mismo rendimiento marginal que los primeros 4.

6. `lcg_skip` **inicial.** Cada hilo ejecuta un salto de $O(\log N)$ pasos al inicio. Con $p = 8$ hay 8 saltos que no existen en la versión secuencial. Su costo es despreciable frente a N/p iteraciones del bucle, pero contribuye al overhead.

III.4. Predicción con 64 núcleos y reflexión sobre p vs. N

Con $f = 0{,}9884$ y $p = 64$:

$$S(64) = \frac{1}{(1 - 0{,}9884) + \dfrac{0{,}9884}{64}} = \frac{1}{0{,}0116 + 0{,}01544} = \frac{1}{0{,}02704} \approx 37{,}0.$$

Con 64 núcleos el speedup máximo sería de unas 37×, muy lejos del ideal lineal de 64×. Esto se debe a que el 1,2 % secuencial ya domina: con 64 núcleos la parte paralela tarda $0{,}9884/64 \approx 1{,}5\,\%$ del tiempo original, comparable al 1,2 % secuencial. Añadir más núcleos más allá de este punto tiene rendimientos decrecientes drásticos.

Aumentar p vs. aumentar N: ¿qué mejora cada uno? Es fundamental distinguir lo que aporta cada variable:

- **Aumentar p con N fijo** reduce el tiempo de cómputo (se reparte el mismo trabajo entre más hilos), pero **no mejora la precisión estadística**: el error sigue siendo $\sigma \propto 1/\sqrt{N}$, que depende solo del número total de muestras, no de cuántos hilos las generen. Con $N = 10^8$ el error típico de $\hat{\pi}$ es $\sim 1{,}6 \times 10^{-4}$ con 1 hilo o con 64.

- **Aumentar N** (tomando más muestras) sí **mejora la precisión**: el error decrece como $1/\sqrt{N}$. Pero con N fijo, necesitamos más tiempo de cómputo; paralelizar permite alcanzar un N mayor en el mismo tiempo de pared.

En resumen: p compra *velocidad*, N compra *precisión*. La paralelización es 'util precisamente porque permite escalar N (y por tanto ganar dígitos) en un tiempo razonable. Pero aumentar p sin aumentar N sólo entrega el mismo resultado más rápido, no un resultado mejor.

III.5. Argumento de Gustafson: por qué Amdahl puede ser pesimista

La ley de Amdahl fija el *tamaño del problema* N y pregunta cuánto se acelera al agregar procesadores. Bajo esa óptica, la fracción secuencial $(1 - f)$ impone un techo insuperable.

Gustafson (1988) observó que en la práctica los usuarios no fijan N: fijan el *tiempo de pared* y, al disponer de más procesadores, **aumentan el problema** que pueden resolver en ese tiempo. Su ley redefine el speedup como

$$S'(p) = (1 - f') + f' \cdot p,$$

donde f' es la fracción paralela *medida en la ejecución real con p procesadores*. Esto es lineal en p, sin techo asintótico.

La diferencia clave es la perspectiva:

- **Amdahl** (problema fijo): "tengo $N = 10^8$ muestras; ¿cuánto más rápido las proceso con 64 núcleos?" Respuesta: 37×, limitado por el 1,2 % secuencial.
- **Gustafson** (tiempo fijo): "tengo 1 segundo de tiempo de pared; ¿cuántas muestras puedo procesar con 64 núcleos?" Con 1 núcleo procesamos $\sim 1{,}6 \times 10^8$ muestras/s; con 64 podríamos procesar $\sim 64 \times 1{,}6 \times 10^8 \approx 10^{10}$, mejorando la precisión de $\hat{\pi}$ en un factor $\sqrt{64} = 8$. La fracción secuencial (crear hilos, reducir p enteros) sigue siendo un número fijo de microsegundos que se vuelve insignificante frente al trabajo paralelo creciente.

Amdahl es pesimista porque asume que el trabajo secuencial crece *proporcionalmente* al total; Gustafson reconoce que la parte secuencial suele ser un costo fijo que se diluye al escalar el problema. Para Monte Carlo, donde siempre hay apetito por más muestras, la visión de Gustafson es la relevante: más núcleos → más N en el mismo tiempo → más precisión.